

Package ‘replicateEverything’

June 29, 2026

Title Tools to Replicate Empirical Research Papers

Version 0.2.0

Description Provides tools for discovering and running computational replications of empirical research papers. The package connects to a replication registry, retrieves replication materials (data, code, and metadata), and reproduces figures and tables from published studies in a standardized workflow.

License MIT + file LICENSE

Encoding UTF-8

Depends R (>= 4.1.0)

Imports jsonlite, yaml, httr, utils, stats

Suggests ggplot2, haven, testthat (>= 3.0.0), knitr, rmarkdown,
DiagrammeR, withr, devtools, remotes

Config/testthat/edition 3

URL <https://github.com/replicate-anything/replicateEverything>,
<https://replicate-anything.github.io/replicateEverything/>

BugReports <https://github.com/replicate-anything/replicateEverything/issues>

Roxygen list(markdown = TRUE)

RoxygenNote 8.0.0

VignetteBuilder knitr

Config/Needs/website pkgdown

NeedsCompilation no

Author Macartan Humphreys [aut],
Cord Masche [aut],
Vernon Washington [aut, cre]

Maintainer Vernon Washington <vermon.washington@wzb.eu>

Contents

replicateEverything-package	3
artifact_available	5
artifact_lookup_candidates	5
call_replication_package	6
clean_replication_source_lines	6

create_replication_template	7
default_artifact_path	7
default_format_name	8
download_registry_file	8
enrich_package_replication_meta	9
ensure_replication_dependencies	9
ensure_replication_package	10
fetch_package_replication_yaml	10
find_replication_entry	11
find_repo	11
format_for_display	12
format_function_name	13
format_script_path	13
format_specified	13
get_analysis_function	14
get_artifact_path	14
get_code	15
get_code_from_package_repo	15
get_doi_metadata	16
get_replication_meta_impl	17
github_package_outdated	17
github_remote_sha	18
infer_result_format	18
installed_package_remote_sha	19
install_replication_package_github	19
is_package_replication	20
list_replications	20
load_artifact	21
load_artifact_file_path	21
load_index	22
load_local_replication_data	22
load_replication_data	23
load_replication_package_path	23
local_artifact_path	24
make_function_name	24
manifest_artifact_paths	25
normalize_doi	25
normalize_html_table	26
package_folder_candidates	26
package_replication_code_urls	27
package_replication_entries	27
package_replication_yaml_urls	28
package_repo_ref	28
package_repo_slug	29
paper_context	29
raw_to_github_browse	30
read_lines_url	30
read_package_repo_source	31
read_yaml_url	31
registry_artifact_rel_paths	32
render_for_display	32
render_replication	33

replicate_paper	33
replication_error_message	34
replication_index_diagnostics	34
replication_object	35
resolve_paper_path	36
resolve_registry_artifact_path	36
resolve_replication_package_path	37
retry_with_missing_package	37
run_replication	38
save_artifact	38
search_papers	39
self_run	40
sibling_monorepo_root	40
sibling_packages_enabled	41
source_replication_functions	41
source_replication_scripts	41
try_render_for_display	42
validate_artifact	42
validate_paper_artifacts	43
validate_replication	44
yaml_url_status	44
Index	45

replicateEverything-package

replicateEverything: Reproduce Empirical Research Results

Description

The replicateEverything package provides tools for discovering and executing computational replications of empirical research papers. It connects to a public replication registry containing metadata, replication scripts, and processed datasets required to reproduce figures and tables from published studies.

Workflow

A typical workflow using the package is:

1. Retrieve metadata for a paper using `get_doi_metadata()`.
2. Search the replication registry using `search_papers()`.
3. Create a template for contributing a replication using `create_replication_template()`.
4. Inspect available replications using `list_replications()`.
5. Run a single replication using `run_replication()`.
6. Reproduce all results from a paper using `replicate_paper()`.

Examples

Retrieve metadata for a paper:

```
get_doi_metadata("10.1177/00491241211036161")
```

Search the registry:

```
search_papers("causal")
```

Create a replication template:

```
create_replication_template("10.1177/00491241211036161")
```

List replications:

```
list_replications("10.1177/00491241211036161")
```

Run a single replication:

```
run_replication("10.1177/00491241211036161", "fig_1")
```

Replicate an entire paper:

```
replicate_paper("10.1177/00491241211036161")
```

Registry

Replication metadata and materials are stored in the public registry: <https://github.com/replicate-anything/registry>.

Author(s)

Maintainer: Vernon Washington <vermon.washington@wzb.eu>

Authors:

- Vernon Washington <vermon.washington@wzb.eu>
- Macartan Humphreys <macartan.humphreys@wzb.eu>
- Cord Masche <cord.masche@wzb.eu>

See Also

Useful links:

- <https://github.com/replicate-anything/replicateEverything>
- <https://replicate-anything.github.io/replicateEverything/>
- Report bugs at <https://github.com/replicate-anything/replicateEverything/issues>

artifact_available	<i>Check whether a precomputed artifact is available</i>
--------------------	--

Description

Check whether a precomputed artifact is available

Usage

```
artifact_available(doi, what, repo = NULL)
```

Arguments

doi	Character. DOI of the paper.
what	Character. Replication identifier (e.g., "fig_1").
repo	Optional repository slug.

Value

Logical scalar.

Examples

```
## Not run:
artifact_available("10.1177/00491241211036161", "fig_1")

## End(Not run)
```

artifact_lookup_candidates	<i>Human-readable list of artifact URLs/paths tried for a replication</i>
----------------------------	---

Description

Human-readable list of artifact URLs/paths tried for a replication

Usage

```
artifact_lookup_candidates(doi, what, repo = NULL, folder = NULL)
```

Arguments

doi	Character. DOI of the paper.
what	Replication identifier.
repo	Optional repository slug.
folder	Optional registry folder name from index.csv.

`call_replication_package`*Call a function from a study replication package*

Description

Call a function from a study replication package

Usage

```
call_replication_package(package, fn, ...)
```

Arguments

<code>package</code>	Package name.
<code>fn</code>	Function name as string.
<code>...</code>	Arguments passed to the function.

`clean_replication_source_lines`*Drop roxygen and export-alias lines from replication source for display*

Description

Drop roxygen and export-alias lines from replication source for display

Usage

```
clean_replication_source_lines(lines)
```

Arguments

<code>lines</code>	Character vector.
--------------------	-------------------

```
create_replication_template
```

Create a replication template

Description

Create a replication template

Usage

```
create_replication_template(doi)
```

Arguments

doi Character. DOI of the paper.

Examples

```
## Not run:
create_replication_template("10.1177/00491241211036161")

## End(Not run)
```

```
default_artifact_path
```

Resolve the expected artifact path for a replication entry

Description

Resolve the expected artifact path for a replication entry

Usage

```
default_artifact_path(rep, what)
```

Arguments

rep A single replication entry from replication.yml.
what Replication identifier.

default_format_name	<i>Default format function name for a replication id</i>
---------------------	--

Description

Default format function name for a replication id

Usage

```
default_format_name(rep_id)
```

Arguments

rep_id	Replication identifier.
--------	-------------------------

download_registry_file	<i>Download a registry or package file to a temp path</i>
------------------------	---

Description

Uses `httr` for HTTP(S) URLs to avoid `download.file()` temp-path failures on some Shiny servers.

Usage

```
download_registry_file(url)
```

Arguments

url	HTTP(S) URL or local path.
-----	----------------------------

Value

Character path to temp file.

enrich_package_replication_meta

Merge replication entries from the study package into registry stub metadata

Description

Merge replication entries from the study package into registry stub metadata

Usage

```
enrich_package_replication_meta(meta, ctx)
```

Arguments

meta	Parsed replication metadata.
ctx	Paper context from <code>paper_context()</code> .

Value

Updated metadata list.

ensure_replication_dependencies

Ensure replication package dependencies are available

Description

Ensure replication package dependencies are available

Usage

```
ensure_replication_dependencies(
  replication_meta,
  paper_meta = NULL,
  install_missing = FALSE
)
```

Arguments

replication_meta	A single replication entry from <code>replication.yml</code> .
paper_meta	Optional paper-level metadata list.
install_missing	Logical. Install missing CRAN packages when TRUE.

Value

Invisibly TRUE.

ensure_replication_package

Load or install a study replication package

Description

Tries, in order: local sibling package (when configured), installed package (upgrade from GitHub when outdated), then fresh GitHub install from package_repo_slug().

Usage

```
ensure_replication_package(package, meta = NULL, ctx = NULL)
```

Arguments

package	R package name.
meta	Parsed replication.yml contents.
ctx	Paper context from paper_context().

fetch_package_replication_yaml

Fetch replication.yml from a study package GitHub repo

Description

Lets the app list tables/figures without installing the package.

Usage

```
fetch_package_replication_yaml(meta, ctx)
```

Arguments

meta	Parsed registry stub or package metadata.
ctx	Paper context from paper_context().

Value

Parsed yaml list or NULL.

`find_replication_entry`*Find a single replication entry by id*

Description

Find a single replication entry by id

Usage

```
find_replication_entry(meta, what)
```

Arguments

<code>meta</code>	Parsed replication metadata.
<code>what</code>	Replication identifier.

`find_repo`*Find the repository for a paper replication*

Description

Looks up the replication registry to determine which repository contains the replication materials for a given DOI.

Usage

```
find_repo(doi)
```

Arguments

<code>doi</code>	Character. DOI of the paper.
------------------	------------------------------

Value

Character string containing the GitHub repository name.

Examples

```
## Not run:  
find_repo("10.1177/00491241211036161")  
  
## End(Not run)
```

format_for_display	<i>Apply an optional format function to an analysis object</i>
--------------------	--

Description

When replication.yml defines format, the analysis object (typically from make_*) is passed to format_* for display. Otherwise the object is returned unchanged.

Usage

```
format_for_display(
  object,
  doi,
  what,
  install_deps = FALSE,
  repo = NULL,
  folder = NULL
)
```

Arguments

object	Analysis output to format.
doi	Character. DOI of the paper.
what	Replication identifier.
install_deps	Logical. Install missing dependencies when TRUE.
repo	Optional repository slug.
folder	Optional registry folder name from index.csv.

Value

Object suitable for display (often an HTML string or ggplot).

Examples

```
## Not run:
result <- render_replication("10.1177/00491241211036161", "fig_1")
format_for_display(replication_object(result), "10.1177/00491241211036161", "fig_1")

## End(Not run)
```

format_function_name	<i>Resolve the format function name for a replication entry</i>
----------------------	---

Description

Resolve the format function name for a replication entry

Usage

```
format_function_name(rep)
```

Arguments

rep	A single replication entry from replication.yml.
-----	--

format_script_path	<i>Resolve optional path to a format script</i>
--------------------	---

Description

Resolve optional path to a format script

Usage

```
format_script_path(rep)
```

Arguments

rep	A single replication entry from replication.yml.
-----	--

format_specified	<i>Check whether a replication entry defines a separate format step</i>
------------------	---

Description

Check whether a replication entry defines a separate format step

Usage

```
format_specified(rep)
```

Arguments

rep	A single replication entry from replication.yml.
-----	--

`get_analysis_function` *Resolve the analysis function from a sourced replication script*

Description

Resolve the analysis function from a sourced replication script

Usage

```
get_analysis_function(env, what, type)
```

Arguments

<code>env</code>	Environment containing sourced definitions.
<code>what</code>	Replication identifier.
<code>type</code>	Replication type (figure or table).

`get_artifact_path` *Get artifact URL or local path for a replication*

Description

Get artifact URL or local path for a replication

Usage

```
get_artifact_path(doi, what, repo = NULL, folder = NULL)
```

Arguments

<code>doi</code>	Character. DOI of the paper.
<code>what</code>	Replication identifier.
<code>repo</code>	Optional repository slug.
<code>folder</code>	Optional registry folder name from <code>index.csv</code> .

Value

Character path or URL, or NULL if no artifact is registered.

Examples

```
## Not run:
get_artifact_path("10.1177/00491241211036161", "fig_1")

## End(Not run)
```

get_code	<i>Retrieve replication code for a paper</i>
----------	--

Description

Returns the analysis script and, when defined, the format script.

Usage

```
get_code(doi, what, repo = NULL, folder = NULL)
```

Arguments

doi	Character. DOI of the paper.
what	Character. Replication identifier (e.g., "fig_1").
repo	Optional repository slug.
folder	Optional registry folder name from index.csv.

Details

For package-backed studies, reads inst/replication_code/*.R from the study package GitHub repo when the package is not installed (same idea as reading code/*.R from the registry repo).

Value

A character vector containing the lines of the replication script(s).

Examples

```
## Not run:
head(get_code("10.1177/00491241211036161", "fig_1"))

## End(Not run)
```

get_code_from_package_repo	<i>Build get_code output for a package-backed study from remote source</i>
----------------------------	--

Description

Build get_code output for a package-backed study from remote source

Usage

```
get_code_from_package_repo(meta, ctx, what, package)
```

Arguments

<code>meta</code>	Parsed replication metadata (with replications list).
<code>ctx</code>	Paper context.
<code>what</code>	Replication id.
<code>package</code>	R package name.

<code>get_doi_metadata</code>	<i>Retrieve metadata for a DOI</i>
-------------------------------	------------------------------------

Description

Fetches bibliographic metadata for a research paper using its DOI. The function queries a DOI resolver and returns basic metadata including the paper title, journal, publication year, and authors.

Usage

```
get_doi_metadata(doi)
```

Arguments

<code>doi</code>	Character. DOI of the paper.
------------------	------------------------------

Details

This function is primarily used by `create_replication_template()` to automatically populate metadata in the `replication.yml` file when creating a new replication.

Value

A list containing:

title Title of the paper

journal Journal name

year Publication year

authors Vector of author names

Examples

```
## Not run:
get_doi_metadata("10.1177/00491241211036161")

## End(Not run)
```

`get_replication_meta_impl`*Fetch replication metadata for a paper*

Description

Fetch replication metadata for a paper

Usage

```
get_replication_meta_impl(doi, repo = NULL, folder = NULL)
```

Arguments

<code>doi</code>	Character. DOI of the paper.
<code>repo</code>	Optional repository slug.
<code>folder</code>	Optional registry folder name from <code>index.csv</code> .

Value

Parsed `replication.yml` contents.

`github_package_outdated`*Whether an installed package lags the GitHub ref*

Description

Whether an installed package lags the GitHub ref

Usage

```
github_package_outdated(package, repo, ref = "main")
```

Arguments

<code>package</code>	Installed package name.
<code>repo</code>	GitHub slug org/repo.
<code>ref</code>	Branch, tag, or commit.

github_remote_sha	<i>Latest commit SHA for a GitHub repo ref</i>
-------------------	--

Description

Uses the GitHub REST API (not remotes) so version checks work on servers where `remotes::remote_sha()` fails on temp paths.

Usage

```
github_remote_sha(repo, ref = "main")
```

Arguments

- | | |
|------|-------------------------|
| repo | GitHub slug org/repo. |
| ref | Branch, tag, or commit. |

infer_result_format	<i>Infer the format of a replication result object</i>
---------------------	--

Description

Infer the format of a replication result object

Usage

```
infer_result_format(object, type)
```

Arguments

- | | |
|--------|--|
| object | Result returned by a replication script. |
| type | Replication type (figure or table). |

Value

Character string describing the result format.

`installed_package_remote_sha`*SHA recorded for a package installed from GitHub*

Description

SHA recorded for a package installed from GitHub

Usage

```
installed_package_remote_sha(package)
```

Arguments

<code>package</code>	Installed package name.
----------------------	-------------------------

Value

Character SHA or NA_character_.

`install_replication_package_github`*Install or upgrade a study package from GitHub*

Description

Install or upgrade a study package from GitHub

Usage

```
install_replication_package_github(package, repo, ref = "main")
```

Arguments

<code>package</code>	R package name.
<code>repo</code>	GitHub slug.
<code>ref</code>	Git ref.

```
is_package_replication
```

Whether replication metadata refers to an installed R package

Description

Whether replication metadata refers to an installed R package

Usage

```
is_package_replication(meta)
```

Arguments

meta Parsed replication.yml contents.

Value

Logical.

```
list_replications
```

List available replications for a paper

Description

List available replications for a paper

Usage

```
list_replications(doi, repo = NULL, folder = NULL)
```

Arguments

doi Character. DOI of the paper.
 repo Optional repository slug.
 folder Optional registry folder name from index.csv.

Value

A list of replication entries from replication.yml.

Examples

```
## Not run:
list_replications("10.1177/00491241211036161")

## End(Not run)
```

load_artifact	<i>Load a precomputed artifact for a replication</i>
---------------	--

Description

Load a precomputed artifact for a replication

Usage

```
load_artifact(doi, what, repo = NULL, folder = NULL)
```

Arguments

doi	Character. DOI of the paper.
what	Character. Replication identifier (e.g., "fig_1").
repo	Optional repository slug.
folder	Optional registry folder name from index.csv.

Value

Artifact contents suitable for display, or NULL.

Examples

```
## Not run:  
load_artifact("10.1177/00491241211036161", "fig_1")  
  
## End(Not run)
```

load_artifact_file_path	<i>Load artifact bytes from a local path or registry URL</i>
-------------------------	--

Description

Load artifact bytes from a local path or registry URL

Usage

```
load_artifact_file_path(path)
```

Arguments

path	Local path or HTTP(S) URL.
------	----------------------------

load_index	<i>Load the replication registry index</i>
------------	--

Description

Load the replication registry index

Usage

```
load_index()
```

Value

A data frame containing replication metadata.

Examples

```
## Not run:
head(load_index())

## End(Not run)
```

load_local_replication_data	<i>Load replication data from local paths (for self-contained scripts)</i>
-----------------------------	--

Description

Used at the bottom of replication scripts so they can be run directly from the paper folder without the package orchestrator.

Usage

```
load_local_replication_data(data_paths, paper_dir = getwd())
```

Arguments

data_paths	Character vector of paths relative to the paper folder.
paper_dir	Paper root directory. Defaults to current working directory.

Value

A data frame, list, or other object.

Examples

```
tmp <- tempfile()
dir.create(tmp)
dir.create(file.path(tmp, "data"))
write.csv(
  data.frame(x = 1:3, y = 4:6),
  file.path(tmp, "data", "example.csv"),
  row.names = FALSE
)
load_local_replication_data("data/example.csv", paper_dir = tmp)
```

load_replication_data *Load replication data files*

Description

Supports CSV, RDS, and Stata DTA files from a local registry checkout or remote raw GitHub URLs.

Usage

```
load_replication_data(data_files, ctx)
```

Arguments

data_files	Character vector of paths relative to the paper folder.
ctx	Paper context from <code>paper_context()</code> .

Value

A data frame, a named list of objects, or NULL.

load_replication_package_path
 Load a package from a local source tree

Description

Load a package from a local source tree

Usage

```
load_replication_package_path(path, package)
```

Arguments

path	Path to package root (contains DESCRIPTION).
package	Expected package name.

local_artifact_path	<i>Get the local artifact file path for a replication, if available</i>
---------------------	---

Description

Get the local artifact file path for a replication, if available

Usage

```
local_artifact_path(doi, what, repo = NULL)
```

Arguments

doi	Character. DOI of the paper.
what	Character. Replication identifier (e.g., "fig_1").
repo	Optional repository slug.

Value

Character path or NULL.

make_function_name	<i>Standard make_* function name for a replication id</i>
--------------------	---

Description

Standard make_* function name for a replication id

Usage

```
make_function_name(what)
```

Arguments

what	Replication identifier.
------	-------------------------

`manifest_artifact_paths`*Artifact paths recorded in artifacts/manifest.json*

Description

Artifact paths recorded in artifacts/manifest.json

Usage

```
manifest_artifact_paths(what, ctx)
```

Arguments

<code>what</code>	Replication id.
<code>ctx</code>	Paper context.

`normalize_doi`*Normalize a DOI*

Description

Cleans and standardizes a DOI string so it can be used consistently across package functions. The function removes common DOI URL prefixes and trims whitespace.

Usage

```
normalize_doi(doi)
```

Arguments

<code>doi</code>	Character. A DOI string or DOI URL.
------------------	-------------------------------------

Value

A cleaned DOI string.

Examples

```
normalize_doi("https://doi.org/10.1177/00491241211036161")
```

normalize_html_table	<i>Decode HTML entities in kableExtra / modelsummary table output</i>
----------------------	---

Description

Decode HTML entities in kableExtra / modelsummary table output

Usage

```
normalize_html_table(html)
```

Arguments

html	Character HTML string.
------	------------------------

Value

Character HTML with common entities decoded for browser display.

Examples

```
normalize_html_table("<table>&nbsp;data</table>")
```

package_folder_candidates	<i>Folder names to check when locating a sibling replication package</i>
---------------------------	--

Description

Folder names to check when locating a sibling replication package

Usage

```
package_folder_candidates(package, meta, ctx)
```

Arguments

package	R package name.
meta	Parsed replication.yml contents.
ctx	Paper context from paper_context().

Value

Character vector of folder names (no duplicates).

`package_replication_code_urls`*Raw GitHub URLs for a study package R source file*

Description

Raw GitHub URLs for a study package R source file

Usage

```
package_replication_code_urls(name, repo, ref = "main")
```

Arguments

<code>name</code>	Base name without .R (e.g. <code>make_figure_1</code>).
<code>repo</code>	GitHub slug.
<code>ref</code>	Git ref.

`package_replication_entries`*Replication entries from a study package when the registry stub omits them*

Description

Replication entries from a study package when the registry stub omits them

Usage

```
package_replication_entries(meta)
```

Arguments

<code>meta</code>	Parsed replication metadata.
-------------------	------------------------------

Value

List of replication/prep entries.

```
package_replication_yaml_urls
```

Raw GitHub URLs for a study package replication.yml

Description

Raw GitHub URLs for a study package replication.yml

Usage

```
package_replication_yaml_urls(repo, ref = "main")
```

Arguments

repo	GitHub slug org/repo.
ref	Branch or tag.

Value

Character vector of raw URLs (tried in order).

```
package_repo_ref
```

Git ref for remotes::install_github()

Description

Git ref for remotes::install_github()

Usage

```
package_repo_ref(meta)
```

Arguments

meta	Parsed replication.yml contents.
------	----------------------------------

Value

Character branch, tag, or commit.

package_repo_slug	<i>Resolve GitHub repo slug for a package-backed replication</i>
-------------------	--

Description

Used when no local sibling package is found. Set `paper.package_repo` or top-level repo in `replication.yml` (GitHub slug, e.g. `replicate-anything/rep_10.1371_journal.pone.0278337`).

Usage

```
package_repo_slug(meta, ctx)
```

Arguments

meta	Parsed replication.yml contents.
ctx	Paper context from <code>paper_context()</code> .

Value

Character repo slug.

paper_context	<i>Build base URLs and paths for a paper in the registry</i>
---------------	--

Description

Build base URLs and paths for a paper in the registry

Usage

```
paper_context(doi, repo = NULL, folder = NULL)
```

Arguments

doi	Character. DOI of the paper.
repo	Optional repository slug. Defaults to <code>find_repo(doi)</code> .
folder	Optional registry folder name from <code>index.csv</code> .

Value

A list with `repo`, `folder`, `base_url`, and optional `local_root`.

raw_to_github_browse	<i>Convert a raw GitHub URL to a browseable /blob/ link</i>
----------------------	---

Description

Convert a raw GitHub URL to a browseable /blob/ link

Usage

```
raw_to_github_browse(url)
```

Arguments

url	Raw raw.githubusercontent.com URL.
-----	------------------------------------

Value

Browse URL or original url.

read_lines_url	<i>Read text lines from a local path or HTTP(S) URL</i>
----------------	---

Description

Read text lines from a local path or HTTP(S) URL

Usage

```
read_lines_url(url)
```

Arguments

url	Character URL or file path.
-----	-----------------------------

Value

Character vector of lines, or empty if not found.

`read_package_repo_source`*Read study package source from GitHub (no install required)*

Description

Read study package source from GitHub (no install required)

Usage

```
read_package_repo_source(name, repo, ref = "main")
```

Arguments

name	Base name without .R (e.g. make_figure_1).
repo	GitHub slug.
ref	Git ref.

Value

Character lines or empty.

`read_yaml_url`*Read yaml from an HTTP(S) URL without writing a temp file*

Description

Avoids `download.file()` temp-path failures on some Shiny servers.

Usage

```
read_yaml_url(url)
```

Arguments

url	Character URL.
-----	----------------

Value

Parsed yaml list or NULL.

```
registry_artifact_rel_paths
```

Relative artifact paths to try under registry papers/<folder>/

Description

Relative artifact paths to try under registry papers/<folder>/

Usage

```
registry_artifact_rel_paths(what, rep = NULL, ctx = NULL)
```

Arguments

what	Replication id.
rep	Optional replication entry.
ctx	Optional paper context (for manifest.json).

```
render_for_display
```

Render a replication and apply formatting for display

Description

Render a replication and apply formatting for display

Usage

```
render_for_display(doi, what, install_deps = FALSE, repo = NULL, folder = NULL)
```

Arguments

doi	Character. DOI of the paper.
what	Character. Replication identifier (e.g., "fig_1").
install_deps	Logical. Install missing CRAN dependencies when TRUE. Defaults to FALSE.
repo	Optional repository slug.
folder	Optional registry folder name from index.csv.

Examples

```
## Not run:
render_for_display("10.1177/00491241211036161", "fig_1")

## End(Not run)
```

render_replication	<i>Render a single replication</i>
--------------------	------------------------------------

Description

Loads data, sources the replication script, and returns a typed result envelope suitable for Shiny display or artifact generation.

Usage

```
render_replication(doi, what, install_deps = FALSE, repo = NULL, folder = NULL)
```

Arguments

doi	Character. DOI of the paper.
what	Character. Replication identifier (e.g., "fig_1").
install_deps	Logical. Install missing CRAN dependencies when TRUE. Defaults to FALSE.
repo	Optional repository slug.
folder	Optional registry folder name from index.csv.

Value

A list with id, type, object, and format.

Examples

```
## Not run:
render_replication("10.1177/00491241211036161", "fig_1")

## End(Not run)
```

replicate_paper	<i>Replicate all results from a paper</i>
-----------------	---

Description

Runs all registered replications (figures and tables) for a given paper.

Usage

```
replicate_paper(doi, install_deps = FALSE)
```

Arguments

doi	Character. DOI of the paper.
install_deps	Logical. Install missing CRAN dependencies when TRUE. Defaults to FALSE.

Value

A named list of replication result envelopes.

Examples

```
## Not run:
replicate_paper("10.1177/00491241211036161")

## End(Not run)
```

```
replication_error_message
```

Format a replication error for user-facing display

Description

Unwraps conditionMessage() and, when present, parent errors and the call that failed.

Usage

```
replication_error_message(x)
```

Arguments

x An error condition or character message.

Value

A single character string suitable for logs or UI.

Examples

```
err <- simpleError("Replication failed", call = quote(run_replication()))
replication_error_message(err)
```

```
replication_index_diagnostics
```

Report where the replication index was sought (for debugging Shiny)

Description

When a package-backed study lists no tables/figures, this shows which replication.yml URLs were checked. The study index should come from the package repo (e.g. replicate-anything/rep_10.1371_journal not only the registry stub under papers/<folder>/).

Usage

```
replication_index_diagnostics(doi, repo = NULL, folder = NULL)
```

Arguments

doi	Character DOI.
repo	Optional registry repo slug from index.csv.
folder	Optional registry folder name.

Value

A list with registry_sources, package_sources, etc.

Examples

```
## Not run:  
replication_index_diagnostics("10.1177/00491241211036161")  
  
## End(Not run)
```

replication_object	<i>Extract a plain object from a replication result envelope</i>
--------------------	--

Description

Extract a plain object from a replication result envelope

Usage

```
replication_object(x)
```

Arguments

x	A replication result list or raw object.
---	--

Value

The underlying replication object.

Examples

```
result <- list(id = "fig_1", object = data.frame(x = 1), class = "replication_result")  
replication_object(result)
```

resolve_paper_path	<i>Resolve the registry folder name for a paper</i>
--------------------	---

Description

Uses the registry index folder column when available, otherwise derives a path from the normalized DOI.

Usage

```
resolve_paper_path(doi)
```

Arguments

doi	Character. DOI of the paper.
-----	------------------------------

Value

Character folder name under papers/.

resolve_registry_artifact_path	<i>Resolve a precomputed artifact under the registry paper folder</i>
--------------------------------	---

Description

Package-backed studies may ship display artifacts in registry/papers/.../artifacts/ even when code lives in the study package.

Usage

```
resolve_registry_artifact_path(what, ctx, rep = NULL, doi = NULL)
```

Arguments

what	Replication id.
ctx	Paper context.
rep	Optional replication entry.
doi	Optional DOI for caching.

`resolve_replication_package_path`*Resolve a local path to a study replication package*

Description

Search order:

1. Explicit path in `paper.package_path` (if it exists)
2. `getOption("replicateEverything.replication_packages")` map
3. Sibling folders under `replication_packages_root` or monorepo root

Usage

```
resolve_replication_package_path(package, meta, ctx)
```

Arguments

<code>package</code>	R package name.
<code>meta</code>	Parsed replication.yml contents.
<code>ctx</code>	Paper context from <code>paper_context()</code> .

Value

Normalized path, or NULL.

`retry_with_missing_package`*Retry an expression after installing a missing package*

Description

Retry an expression after installing a missing package

Usage

```
retry_with_missing_package(expr, install_missing = FALSE, max_attempts = 2)
```

Arguments

<code>expr</code>	Expression to evaluate.
<code>install_missing</code>	Logical. Whether package installation is allowed.
<code>max_attempts</code>	Maximum number of attempts.

run_replication	<i>Run a single replication</i>
-----------------	---------------------------------

Description

Executes a specific replication (figure or table) for a paper.

Usage

```
run_replication(doi, what, install_deps = FALSE)
```

Arguments

doi	Character. DOI of the paper.
what	Character. Replication identifier (e.g., "fig_1").
install_deps	Logical. Install missing CRAN dependencies when TRUE. Defaults to FALSE.

Value

The underlying replication object.

Examples

```
## Not run:
run_replication("10.1177/00491241211036161", "fig_1")

## End(Not run)
```

save_artifact	<i>Save a replication result as an artifact file</i>
---------------	--

Description

Save a replication result as an artifact file

Usage

```
save_artifact(
  result,
  output_dir,
  doi = NULL,
  repo = NULL,
  folder = NULL,
  install_deps = FALSE
)
```

Arguments

result	A replication result envelope from <code>render_replication()</code> .
output_dir	Directory in which to write the artifact.
doi	Optional DOI; required to apply a registered <code>format_*</code> step.
repo	Optional repository slug.
folder	Optional registry folder name.
install_deps	Logical; passed to <code>format_for_display()</code> .

Value

Invisibly the output file path.

Examples

```
tmp <- tempfile()
dir.create(tmp)
result <- structure(
  list(
    id = "tab_1",
    type = "table",
    object = data.frame(x = 1:2, y = 3:4),
    format = "data.frame",
    meta = list(id = "tab_1")
  ),
  class = "replication_result"
)
save_artifact(result, tmp)
```

search_papers

Search replicated papers

Description

Searches the replication registry for papers matching a keyword in the title or author fields.

Usage

```
search_papers(query)
```

Arguments

query	Character string used to search paper titles or authors.
-------	--

Value

A filtered data frame of matching papers.

Examples

```
## Not run:
search_papers("causal")

## End(Not run)
```

self_run

Run a replication script footer when executed directly

Description

Scripts normally use an if/else footer: `generate_figure <- make_fig_1(read.csv(...))` when run directly, and `generate_figure <- make_fig_1` when sourced by the package.

Usage

```
self_run(make_fn, data_paths, paper_dir = getwd())
```

Arguments

make_fn	Function that accepts data.
data_paths	Paths relative to the paper folder.
paper_dir	Paper root directory.

Examples

```
## Not run:
# At the bottom of a replication script in code/:
# self_run(make_fig_1, "data/fig_1.csv")

## End(Not run)
```

sibling_monorepo_root *Parent directory of the registry when developing in a monorepo*

Description

Parent directory of the registry when developing in a monorepo

Usage

```
sibling_monorepo_root()
```

Value

Character path or NULL.

`sibling_packages_enabled`*Whether to resolve study packages from sibling monorepo folders*

Description

Off by default on servers; enable via `local.R` or `replicateEverything.use_sibling_packages`.

Usage

```
sibling_packages_enabled()
```

`source_replication_functions`*Load only function definitions from a replication script*

Description

Skips the self-run footer so top-level execution (data load + pipe) does not run when the package sources the file.

Usage

```
source_replication_functions(path, env, install_deps = FALSE)
```

Arguments

<code>path</code>	Path to an R script.
<code>env</code>	Environment in which to define functions.
<code>install_deps</code>	Logical. Passed to dependency retry helper.

`source_replication_scripts`*Source replication and optional format scripts into an environment*

Description

Source replication and optional format scripts into an environment

Usage

```
source_replication_scripts(  
  rep,  
  ctx,  
  env,  
  install_deps = FALSE,  
  include_format = TRUE  
)
```

```
try_render_for_display
```

Run a replication and return a result or error object

Description

Like `render_for_display` but never throws; failures are returned as `simpleError` objects.

Usage

```
try_render_for_display(doi, what, install_deps = FALSE, repo = NULL)
```

Arguments

<code>doi</code>	Character. DOI of the paper.
<code>what</code>	Character. Replication identifier (e.g., "fig_1").
<code>install_deps</code>	Logical. Install missing CRAN dependencies when TRUE. Defaults to FALSE.
<code>repo</code>	Optional repository slug.

Value

A replication result list, a display-ready object, or an error.

Examples

```
## Not run:
try_render_for_display("10.1177/00491241211036161", "fig_1")

## End(Not run)
```

```
validate_artifact
```

Validate that a precomputed artifact exists

Description

Validate that a precomputed artifact exists

Usage

```
validate_artifact(doi, what, repo = NULL)
```

Arguments

<code>doi</code>	Character. DOI of the paper.
<code>what</code>	Character. Replication identifier (e.g., "fig_1").
<code>repo</code>	Optional repository slug.

Value

Invisibly TRUE on success.

Examples

```
## Not run:  
validate_artifact("10.1177/00491241211036161", "fig_1")  
  
## End(Not run)
```

validate_paper_artifacts

Validate all artifacts for a paper

Description

Validate all artifacts for a paper

Usage

```
validate_paper_artifacts(doi, repo = NULL)
```

Arguments

doi	Character. DOI of the paper.
repo	Optional repository slug.

Value

Invisibly TRUE if every replication has an artifact.

Examples

```
## Not run:  
validate_paper_artifacts("10.1177/00491241211036161")  
  
## End(Not run)
```

validate_replication	<i>Validate that a replication can be rendered</i>
----------------------	--

Description

Validate that a replication can be rendered

Usage

```
validate_replication(doi, what, install_deps = FALSE)
```

Arguments

doi	Character. DOI of the paper.
what	Replication identifier.
install_deps	Logical. Passed to render_replication().

Value

Invisibly TRUE on success.

Examples

```
## Not run:
validate_replication("10.1177/00491241211036161", "fig_1")

## End(Not run)
```

yaml_url_status	<i>Probe a yaml URL and return a short status string</i>
-----------------	--

Description

Probe a yaml URL and return a short status string

Usage

```
yaml_url_status(url)
```

Arguments

url	Local path or HTTP(S) URL.
-----	----------------------------

Index

* internal

- artifact_lookup_candidates, 5
- call_replication_package, 6
- clean_replication_source_lines, 6
- default_artifact_path, 7
- default_format_name, 8
- download_registry_file, 8
- enrich_package_replication_meta, 9
- ensure_replication_dependencies, 9
- ensure_replication_package, 10
- fetch_package_replication_yaml, 10
- find_replication_entry, 11
- find_repo, 11
- format_function_name, 13
- format_script_path, 13
- format_specified, 13
- get_analysis_function, 14
- get_code_from_package_repo, 15
- get_replication_meta_impl, 17
- github_package_outdated, 17
- github_remote_sha, 18
- infer_result_format, 18
- install_replication_package_github, 19
- installed_package_remote_sha, 19
- is_package_replication, 20
- load_artifact_file_path, 21
- load_replication_data, 23
- load_replication_package_path, 23
- local_artifact_path, 24
- make_function_name, 24
- manifest_artifact_paths, 25
- normalize_doi, 25
- package_folder_candidates, 26
- package_replication_code_urls, 27
- package_replication_entries, 27
- package_replication_yaml_urls, 28
- package_repo_ref, 28
- package_repo_slug, 29
- paper_context, 29
- raw_to_github_browse, 30
- read_lines_url, 30
- read_package_repo_source, 31

- read_yaml_url, 31
- registry_artifact_rel_paths, 32
- resolve_paper_path, 36
- resolve_registry_artifact_path, 36
- resolve_replication_package_path, 37
- retry_with_missing_package, 37
- sibling_monorepo_root, 40
- sibling_packages_enabled, 41
- source_replication_functions, 41
- source_replication_scripts, 41
- yaml_url_status, 44

* package

- replicateEverything-package, 3

- artifact_available, 5
- artifact_lookup_candidates, 5

- call_replication_package, 6
- clean_replication_source_lines, 6
- create_replication_template, 7

- default_artifact_path, 7
- default_format_name, 8
- download_registry_file, 8

- enrich_package_replication_meta, 9
- ensure_replication_dependencies, 9
- ensure_replication_package, 10

- fetch_package_replication_yaml, 10
- find_replication_entry, 11
- find_repo, 11
- format_for_display, 12
- format_function_name, 13
- format_script_path, 13
- format_specified, 13

- get_analysis_function, 14
- get_artifact_path, 14
- get_code, 15
- get_code_from_package_repo, 15
- get_doi_metadata, 16
- get_replication_meta_impl, 17
- github_package_outdated, 17

github_remote_sha, 18

infer_result_format, 18

install_replication_package_github, 19

installed_package_remote_sha, 19

is_package_replication, 20

list_replications, 20

load_artifact, 21

load_artifact_file_path, 21

load_index, 22

load_local_replication_data, 22

load_replication_data, 23

load_replication_package_path, 23

local_artifact_path, 24

make_function_name, 24

manifest_artifact_paths, 25

normalize_doi, 25

normalize_html_table, 26

package_folder_candidates, 26

package_replication_code_urls, 27

package_replication_entries, 27

package_replication_yaml_urls, 28

package_repo_ref, 28

package_repo_slug, 29

paper_context, 29

raw_to_github_browse, 30

read_lines_url, 30

read_package_repo_source, 31

read_yaml_url, 31

registry_artifact_rel_paths, 32

render_for_display, 32, 42

render_replication, 33

replicate_paper, 33

replicateEverything
 (replicateEverything-package),
 3

replicateEverything-package, 3

replication_error_message, 34

replication_index_diagnostics, 34

replication_object, 35

resolve_paper_path, 36

resolve_registry_artifact_path, 36

resolve_replication_package_path, 37

retry_with_missing_package, 37

run_replication, 38

save_artifact, 38

search_papers, 39

self_run, 40

sibling_monorepo_root, 40

sibling_packages_enabled, 41

source_replication_functions, 41

source_replication_scripts, 41

try_render_for_display, 42

validate_artifact, 42

validate_paper_artifacts, 43

validate_replication, 44

yaml_url_status, 44